

Recap

TCP congestion control:

Slow start, Congestion Avoidance (AIMD), Fast recovery

* Signals considered by TCP for congestion control

- 1) Timeout
- 2) 3 dup Acks

* Slow start

1. set $cwnd = 1MSS$
2. send 'cwnd' bytes during transmission round
3. If all Acks arrive on time (ie, No timeout during round)
 - a. If $cwnd < ssthresh$; double cwnd (max val: ssthresh)
 - b. else Enter congestion avoidance phase (AIMD)
4. If timeout occurs for some segment 's'
 - a. retransmit S
 - b. set $ssthresh = \frac{cwnd}{2}$

* Congestion avoidance (AIMD)

1. For any given transmission round; if all Acks arrive on time, $cwnd = cwnd + 1MSS$
2. If timer expires, $ssthresh = cwnd/2$; enter Slow start.

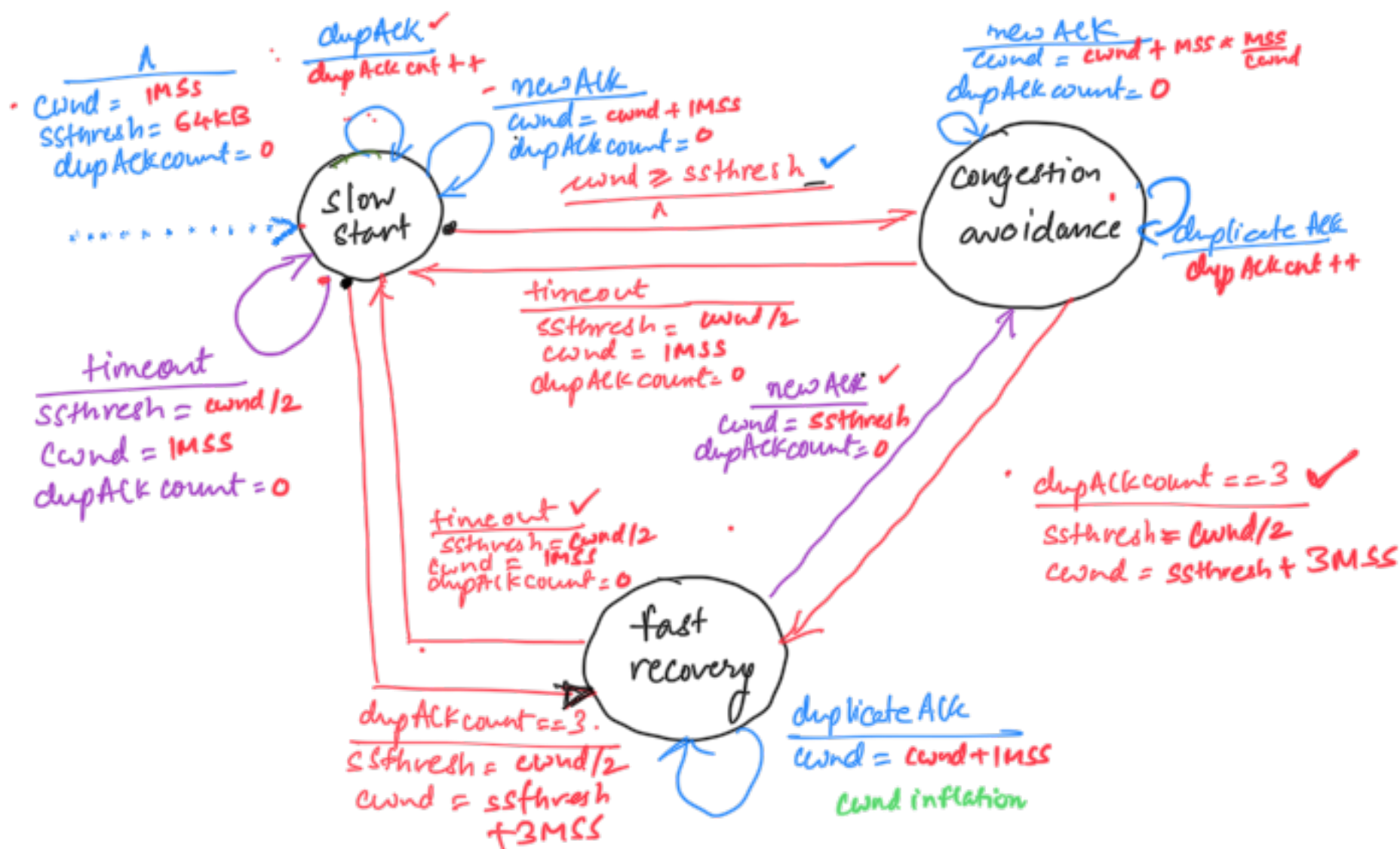
* Fast recovery (If 3 dup Acks recd. TCP Tahoe RT, ssthresh & " TCP Reno retransmits S & does fall-)

- For TCP Reno
1. $ssthresh = cwnd/2$
 2. $cwnd = ssthresh + 3MSS$ for 3 dup Acks cwnd inflation
 3. For every incoming dup Ack thereafter, $cwnd = cwnd + 1MSS$ ie, allowed to send extra data
 4. When new Ack arrives, $cwnd = ssthresh$; switch to congestion avoidance phase

TCP Congestion Control

* TCP Reno

TCP Tahoe ↳ X fast recovery



MSS = 64KB
cwnd = 640KB

window size = 10

Cong.
Avoidance $\Rightarrow$

Window size decreases
 $cwnd = cwnd + 1 \text{ MSS}$
1 seg

MSS

1
newAck $\Rightarrow cwnd = cwnd + \frac{1 \text{ MSS}}{\# \text{ segs in window}}$

$$i = i + \frac{1 \text{ MSS}}{\frac{cwnd}{\text{MSS}}}$$

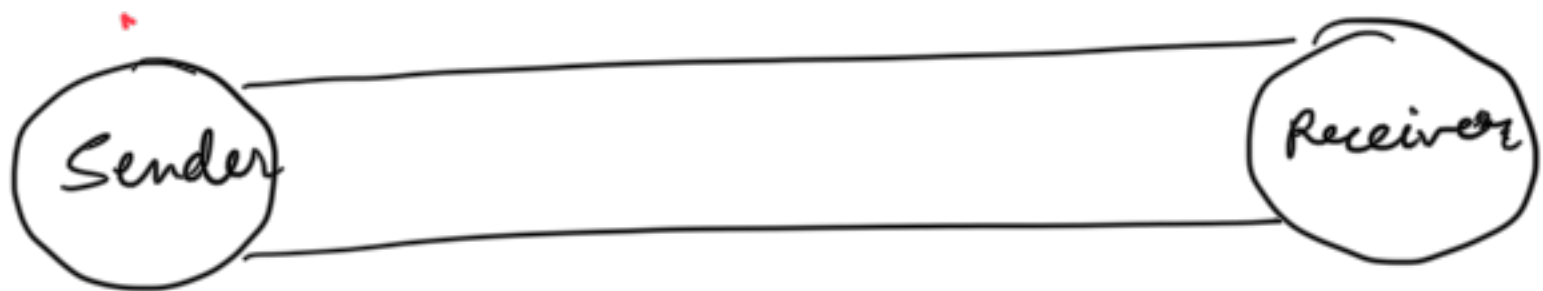
newAck $\Rightarrow$

$$cwnd = cwnd + \text{MSS} \times \frac{\text{MSS}}{cwnd}$$

- Typical TCP behavior: AIMD used to probe usable bandwidth



Congestion window, cwnd tries to estimate BDP



PIPE FULL :

PIPE NOT FULL:

PIPE FULLER:

$$BDP = \text{Bottleneck Bandwidth} * RTT$$